

Full-Stack Assignment

Unified Org Workspace (Ticketing + PR/Audit Console, JWT-Linked Identity)

Objective

Build a unified organization workspace comprising two dashboards - a **Support Hub (ticketing)** and a **Review & Audit Console (PR workflow)** - that share a single identity layer and feel like one coherent product to the end user.

The assignment is designed to assess:

- Your ability to design and implement a **secure, multi-tenant full-stack** system with **shared authentication** across applications.
- Your skill in enforcing **tenant isolation**, **cross-org access controls**, and **append-only audit logging**.
- Your product intuition for session management, **role-based access**, and real-world security constraints (**BOLA**, token revocation, password handling).

Frontend must be **Next.js** or **React.js**. Backend and database are your choice, kept focused on supporting the identity, isolation, and audit requirements described below.

Core Functional Requirements

1. Shared Identity & Authentication Layer

- Email and password **authentication**.
- A central Identity/Org service acts as the **single source of truth** for users and organizations. Both dashboards read from this service; neither dashboard owns this data independently.
- **Session sync mechanism** - Both dashboards live under one parent domain
- **Org switcher**: A user belonging to multiple organizations must be able to switch active org context.
- **Logout-everywhere**: Log out from Both the dashboards.

2. Dashboard 1 - Support Hub

- Ticket **CRUD operations**, comments, attachments, and status management.
- Tenant isolation enforced at the query layer - the **BOLA (Broken Object Level Authorization)** test is non-negotiable.
- **Append-only audit log** recorded for every mutation.
- Per-tenant feature flags.
- A ticket can be shared with users from another organization. Shared users can view and participate in that ticket only — they cannot access any other tickets or data from the sharing organization.
- **Reviewer access**: Users with the Reviewer / Approver role must be able to access and review tickets in the Support Hub, in addition to their PR review responsibilities in Dashboard 2.

3. Dashboard 2 - Review & Audit Console

- **PR entity** comprising title, description, status (draft → in-review → approved/rejected → merged), linked organization, author, and a list of reviewers.
- Approval workflow supporting multiple reviewers, a configurable **"requires N approvals"** rule, and the ability for reviewers to request changes.

- **Versioning:** Every edit made after review has started creates a new version, with a diff view against the previous version.
- **Unified audit viewer:** A searchable, filterable timeline spanning both dashboards - filterable by organization, user, date range, and action type, and exportable as CSV.
- **Reviewer access:** Users with the Reviewer / Approver role must have access to both the PR review workflow and the unified audit trail viewer.

4. Cross-Org Collaboration

- Organizations can **connect** with partner organizations. A connection must be requested, approved, and revocable by either side.
- Once connected, either organization may **share individual tickets or PRs** with the partner — one item at a time, never full access to the other organization's workspace.
- External users from a partner organization can only see what has been explicitly shared with them.
- External users have **restricted access**: they can view and comment only — no editing, deleting, or accessing unshared content.

5. AI Progress Tracker

- Each user receives a **personalized digest scoped** to them (e.g. "You have 4 tickets assigned to you, 1 overdue" or "2 PRs are waiting on your review; oldest is 3 days idle").
- Digests are **delivered** on a **schedule via a background job** at regular, configurable intervals (e.g. every morning, or every N hour), not computed on page load.
- In-app notification bell at minimum.
- A user's digest may only include data from their own organization plus anything explicitly shared with them.

6. Access Control & Security

The system recognizes the following roles, each with a distinct scope of access:

- **Org Admin** - Full control within their own organization, across both dashboards.
- **Support Agent** - Dashboard 1 only; manages tickets for their organization.
- **Reviewer / Approver** - Access to both dashboards: reviews and approves pull requests in Dashboard 2, reviews tickets in the Support Hub (Dashboard 1), and has access to the unified audit trail viewer.
- **Cross-Org Guest** - Limited, explicitly granted access to specific shared tickets or PRs from a partner organization - no access to anything else within that organization.
- **Platform Super Admin** - Manages organizations, cross-org connections, and global platform settings.

Additional security requirements:

- Every query enforces organization scoping at the **query layer**, verified by **automated tests**, extended to cover the cross-org share path.
- A single, unified, append-only audit trail covers both dashboards.
- **Role-Based Access Control (RBAC)**: organization-level roles layered with app-specific roles.
- All cross-org sharing actions are audit-logged.

7. Non-Functional Requirements

- **Isolation** must hold even under **direct API calls** with manipulated IDs. An automated test covering this scenario is required.
- AI summaries must never leak cross-org data. A dedicated test covering this scenario is required.
- The audit log must be appended only, enforced at the database permission level.
- Both dashboards must be independently deployable.

Extended (Optional) Scope

To showcase creative full-stack problem-solving, you may extend with:

- **GitHub integration** to mirror the status of a real **PR via webhook**.
- **Email/push delivery** for AI digests and notifications.
- Advanced audit analytics or anomaly detection.
- Custom org **onboarding flows** or admin console.

Use your design judgment to make the interface feel fluid, modern, and intuitive.

Technology Expectations

Architecture

- Well-structured module boundaries: Identity, Tickets, PRs, and Audit.
- Separation of auth logic, business logic, and data access layers.
- Shared component library across both dashboards.
- Automated tests for isolation, session sync, and token revocation.

Suggested Stack

- **Frontend:** Next.js + React + Tailwind CSS, with a shared component library across both dashboards.
- **Backend:** Node.js / Express.js.
- **Database:** PostgreSQL via Prisma.
- **Cache / Session:** Redis.
- **AI:** Any LLM API.

Deliverables

- **Source Code Repository (GitHub link)**
 - Include basic seed data or a mock backend if required - at minimum, one sample org, one accepted cross-org connection, and a handful of sample tickets/PRs to demonstrate isolation and sharing behavior out of the box.
- **Hosted Project Public URL**
 - Provide a public URL where both dashboards (Support Hub and Review & Audit Console) can be accessed and evaluated, including test credentials for at least two organizations.
- **Documentation Folder (/docs)**
 - **System architecture diagram** (Identity/Org service, Dashboard 1, Dashboard 2).
 - **Setup guide** (local run).
 - Known limitations and future improvements.
- **Short Demo Video (~2 min)**

PS: You may add the agentic ide/LLM used (if any) for coding with reasoning, pros and cons in the documentation

Evaluation Focus

This assignment tests your ability to:

- Architect a secure, multi-tenant system with shared identity across applications.
- Design correct session synchronization and token lifecycle management.
- Enforce tenant and cross-org isolation with automated verification.
- Implement production-grade audit logging and role-based access control.
- Communicate your reasoning clearly and work independently under open constraints.